

Evaluating Virtual Memory Strategies for Diverse Computing Environments



CSE 4501 : Operating Systems

Assigned to :

Saadman Sakib

ID: 210042165

Program : BSc. Engg. SWE

Department of Computer Science and Engineering

Islamic University of Technology

Evaluating Virtual Memory Strategies for **Desktop, Mobile & Cloud** Computing Environments

Abstract :

This assignment analyzes how different virtual memory techniques are suited for various computing environments. The focus is on three distinct platforms—**desktop systems, mobile devices, and cloud servers**—each with unique performance requirements and hardware constraints. Virtual memory concepts like paging, segmentation, and page replacement algorithms for these environments are explored. The study suggests appropriate virtual memory strategies configured to each use case. Trade-offs between speed, efficiency, and resource availability were taken into account. A comparison is made between the proposed strategies, and those typically implemented in real-world systems. The analysis also connects these practices to theory discussed in the Operating Systems course.

1. Introduction :

1.1 : Definition of Virtual Memory

Virtual memory is a memory management technique used by operating systems to give the appearance of a large, continuous block of memory to applications, even if the physical memory (RAM) is limited. It allows larger applications to run on systems with less RAM.^[1]

1.2 : Selected Environments

- Desktop Computers
- Mobile Devices
- Cloud Servers

Desktop computers prioritize performance and user responsiveness, mobile devices focus on energy efficiency and limited physical memory, and cloud servers demand scalability and resource sharing across virtual machines.

2. Environment Profiles & Requirements

Environment	Key Characteristics	Memory Constraints and Needs
Desktop	General Purpose Computing with multi-window apps	Moderate memory, performance-focused, multitasking essential
Mobile Devices	Battery-operated, less RAM, app sandboxing	Limited RAM, power-sensitive, frequent app switching
Cloud Servers	Hosts VMs/containers, scales dynamically, high uptime	Scalability, high concurrency, efficient sharing, reliability

3. Recommended Virtual Memory Strategies

3.1 : Desktop Computers

- Strategy: Demand Paging ^[2] + LRU or WS Clock Page Replacement
- Why:
 1. Supports multitasking without consuming too much physical RAM.
 2. LRU approximations (like the Clock algorithm) balance performance and overhead.
- Segmentation: Can be used for stack/heap separation. ^[2]
- Real-world match: Windows and Linux use demand paging and Clock/LRU approximations.

3.2 : Mobile Devices

- Strategy: Swapping Avoidance + App Freezing + Compressed RAM (zRAM in Android)
- Why:
 1. Flash storage wear-out and slow I/O make swapping undesirable.
 2. OS freezes background apps and keeps only essential data in compressed RAM.
- Android uses Low Memory Killer or newer **OOM Adjuster** ^[3] for aggressive memory management.
- Segmentation: Not used; mostly paging + app sandboxing.
- Real-world match: Android uses **zRAM** ^[4] + aggressive process killing under memory pressure.

3.3 : Cloud Servers

- Strategy: Shared Memory + Ballooning^[5] (for VMs) + Large Pages (HugePages)
- Why:
 1. Minimize duplication using shared libraries/pages across VMs/containers.
 2. Ballooning lets hypervisors dynamically manage guest VM memory.
- HugePages reduce TLB misses and overhead for large in-memory datasets.
- Page Replacement: Adaptive algorithms like NFU variants or custom implementations (e.g., in KVM or Xen).
- Real-world match: KVM, Xen, and cloud platforms like AWS, GCP use ballooning and HugePages.

4. Theoretical Justification & Textbook Concepts

Concept	Application in selected strategy
Demand Paging	Used in desktop and mobile; avoids loading entire programs into memory
Page Replacement	LRU/Clock in desktops, app priority-based replacement in mobile, NFU/2Q in servers
Segmentation	Minimal use today; mostly used in legacy or controlled environments
Swapping	Still viable in desktops/servers, avoided in mobiles
TLB Optimization	HugePages in servers optimize TLB usage

5. Comparison with Current Implementations

Environment	Strategy Proposed	Real-world Match
Desktop	Paging + LRU/Clock	Windows/Linux default
Mobile	Compressed RAM + Freezing	Android uses zRAM, OOM killer
Cloud Server	HugePages + Ballooning	Used in KVM, Xen, Kubernetes, etc.

References :

-
- [1]<https://www.geeksforgeeks.org/virtual-memory-in-operating-system/>
 - [2]<https://os.ecci.ucr.ac.cr/slides/Abraham-Silberschatz-Operating-System-Concepts-10th-2018.pdf>
 - [3]<https://cs.android.com/android/platform/superproject/+master:frameworks/base/services/core/java/com/android/server/am/OomAdjuster.java;l=52>
 - [4]<https://developer.android.com/topic/performance/memory-management>
 - [5]https://www.alibabacloud.com/tech-news/a/memory_virtualization/4okq5h7imw2-memory-ballooning-demystified

